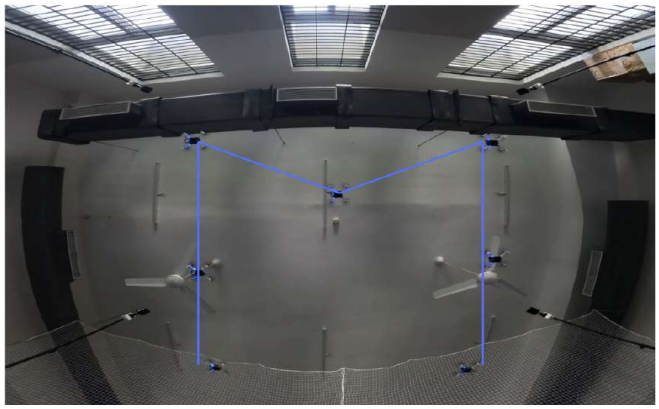
DEPARTMENT OF AEROSPACE ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Pursuit Based Guidance for Multi-Agent Formation Control



A Project Report

Submitted by

HARISH S

AE21B021

For the award of the degree

Of

BACHELOR OF TECHNOLOGY

June 2025

Errors, like straws, upon the surface flow;
He who would search for pearls must dive
below.

– John Dryden

To all those who have supported and guided me throughout my journey.

THESIS CERTIFICATE

This is to undertake that the Project Report titled **PURSUIT BASED GUIDANCE FOR MULTI-AGENT FORMATION CONTROL**, submitted by me to the Indian Institute of Technology Madras, for the award of **Bachelor of Technology**, is a bona fide record of the research work done by me under the supervision of **Dr. Satadal Ghosh**. The contents of this Project Report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Chennai 600036 **Harish S**

Date: June 2025

Dr. Satadal Ghosh
Research advisor
Associate Professor
Department of Aerospace Engineering -
IIT Madras

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who have supported and guided me throughout this journey of my bachelor's project. First and foremost, I would like to thank my project supervisor, Prof. Satadal Ghosh, for their invaluable guidance, encouragement, and unwavering support throughout the entire process. Their insightful feedback and expertise have been instrumental in shaping this project. I would like to extend my thanks to my friends at the lab, Vivek, Harishwar, Ashwin and Shubham for their help on different technical aspects of the project, engaging discussions, and moral support, which have been a constant source of motivation. My sincere appreciation goes to the Aerospace Department, for providing me with the necessary resources and facilities to carry out this research successfully. Finally, I am forever indebted to my family for their unconditional love, support, and encouragement throughout my academic journey. Their belief in me has been a driving force, enabling me to overcome challenges and achieve my goals.

ABSTRACT

This project presents the development and implementation of a multi-agent drone formation control system based on the Crazyflie platform and the Crazyswarm ROS framework. The system facilitates coordinated flight of seven quadrotor drones within a constrained workspace, achieving precise formation control and collision avoidance. A global planner manages formation dynamics, while collision avoidance is accomplished using Artificial Potential Field (APF) methods. To optimize responsiveness and reduce computational overhead, Detective Derivative Pursuit (DDP) is integrated, enabling drones to infer and track dynamic or virtual targets based on observed relative motion changes. The effectiveness of this hybrid control architecture is validated through successful formation of the drones into predefined letter shapes ('I', 'T', 'M') and lateral movement, with consistent results demonstrated in both simulation and real-world flight experiments.

CONTENTS

LIST OF TABLES

Table	Caption	Page

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

Control of multi-robot systems has attracted much research in recent years owing to its potential for solving problems that are difficult or nearly impossible for a single robot. Formation control is one of the common areas where multi-robot systems are involved. In this scenario, two or more robots move to a desired posture to attain formation or move in a defined shape to a desired destination

Unmanned Aerial Vehicles(UAVs) particularly have rapidly become essential tools in various applications such as surveillance, search and rescue, environmental monitoring, and even delivery systems. Sometimes these missions are complex and need more resources to complete efficiently. Hence, the need for multiple drones to work together, called swarm of drones, arises. This is where Formation Control, managing the positions and orientations of multiple drones at the same time becomes crucial.

Formation flight control strategies can broadly be classified into **centralized** and **decentralized** approaches based on the type of computation involved.

Centralized Control

In centralized schemes, a central processing unit—typically a ground-based base station or a designated UAV with higher computational capacity—is responsible for managing the coordination of the entire formation. This central entity gathers information from all other agents and makes global decisions accordingly. While this approach simplifies coordination and ensures consistency, it introduces significant drawbacks, including limited robustness, increased energy consumption, and a single point of failure. Any malfunction in the central unit can compromise the entire formation's operation.

Decentralized Control

In contrast, decentralized (or distributed) schemes eliminate the need for a central coordinator. Each agent within the formation is equipped with local processing capabilities and makes decisions based on local observations and peer-to-peer communication. This architecture addresses the computational and communication bottlenecks inherent to centralized systems and offers improved robustness and scalability. However, it poses greater challenges in terms of system design, coordination logic, and real-time implementation.

Recent research highlights the growing complexity of UAV formation control. Ouyang et al. (2023) provided a comprehensive review of UAV swarm formation control, noting that distributed methods are often superior but centralized approaches excel in scenarios requiring precise coordination [1]. A study by Wang et al. (2024) introduced a UAV formation control algorithm combining improved artificial potential fields and consensus, effectively addressing collision avoidance and communication maintenance [2].

There are three major problems to be addressed in the context of formation control. The first is the formation generation task, where agents starting from random initial positions are guided to form the desired topology. The second is maintaining the shape of the formation during operation, ensuring that the relative configuration between agents is preserved. The third is formation reconfiguration, which becomes necessary when the formation faces disruptions such as obstacle avoidance, agent failures, or loss of communication. In such scenarios, the formation must adapt and reorganize itself to suit the new environment. Different control strategies—both centralized and distributed—have been proposed to tackle these challenges in formation control.

1.1 VIRTUAL STRUCTURE

The virtual structure approach treats the UAV formation as a rigid body, with agents maintaining fixed positions relative to a virtual reference point. Cai et al. (2022)

applied this method to UAV formations, combining it with artificial potential fields to achieve cooperative control under constraints [3]. While effective for precise geometric configurations, this approach struggles in dynamic environments due to its rigidity and high computational demands.

1.2 CONSENSUS BASED APPROACH

Consensus-based methods ensure UAVs converge to a common state through local interactions, often modeled using graph theory. Zhang et al. (2019) combined consensus theory with leader-follower principles to address communication delays in large AUV groups, improving scalability [4]. While effective for decentralized systems, these methods face challenges in convergence time and robustness under complex topologies.

1.3 POTENTIAL FIELD BASED APPROACH

The potential field approach uses virtual attractive and repulsive forces to guide UAVs toward desired positions while avoiding collisions. Wang et al. (2024) proposed a UAV formation control algorithm based on an improved artificial potential field and consensus, addressing internal collision avoidance and communication distance maintenance [2]. This method excels in flexible navigation but can suffer from local minima, where UAVs get trapped in suboptimal configurations. This method performs considerately better in centralized environments and can be used for collision avoidance among agents

1.4 GRAPH BASED APPROACH

Graph-based methods model UAV systems as graphs, with nodes representing agents and edges denoting communication links. Mesbahi and Egerstedt (2010) utilized graph theory to enable flexible topologies and distributed control for multi-agent systems [5]. While powerful for modeling complex interactions, these methods require robust communication networks and can be computationally intensive.

1.5 LEADER-FOLLOWER

The leader-follower approach has been extensively studied for UAVs due to its intuitive design. Li et al. (2023) developed a leader-follower formation control method for lightweight UAVs using novel active disturbance rejection control, achieving robust performance under constraints [6]. To address uncertainties, Nguyen and Hong (2019) proposed a robust adaptive formation control (RAFC) for quadcopters, incorporating nonlinear models to handle external disturbances [7]. Their results showed improved stability, but computational complexity remains a challenge for resource-constrained UAVs.

Centralized leader-follower variants enhance robustness by coordinating all UAVs through a central controller. For example, a Neural Adaptive Prescribed Performance Controller (NA-PPC) was introduced to enable precise trajectory tracking under external disturbances using Radial Basis Function Neural Networks (RBFNN) [8]. Similarly, a hierarchical formation control strategy was proposed for UAVs with system uncertainties, employing sliding mode neural-based observers to estimate nonlinear dynamics, thereby ensuring uniform ultimate boundedness of tracking errors [9].

While centralized approaches offer strong coordination, they can introduce significant computational overhead. This limitation is addressed by the *Detective Deviative Pursuit (DDP)* strategy [10] proposed by Segal et al. DDP is a pursuit-based variation of the leader-follower method that involves lower number of parameters for each agent at every time step. As a result, it provides a scalable solution with low computational complexity.

1.6 MOTIVATION & OBJECTIVE

Motivation: Coordinated control of multi-agent Unmanned Aerial Vehicles (UAVs) is a rapidly evolving field driven by the increasing demand for automation in surveillance, search and rescue, mapping, and logistics. In such missions, the ability of drones to maintain formations while navigating dynamic and potentially constrained environments

is critical. Traditional centralized methods, while effective in providing tight coordination, suffer from computational bottlenecks and single points of failure. On the other hand, distributed approaches, though scalable and robust, introduce complexities in design and synchronization.

Within this context, the Detective Deviative Pursuit (DDP) strategy presents an efficient leader-follower formation mechanism with reduced computational overhead. However, it assumes an already-formed configuration and is not suitable for scenarios where drones begin from random, unstructured positions. To bridge this gap, Artificial Potential Field (APF) methods offer a viable solution for initial formation generation and collision avoidance, albeit with higher computational demands.

The integration of APF for formation generation with DDP for formation maintenance and maneuvering provides a balanced solution—leveraging the strengths of both methods to create a robust, scalable, and computationally efficient system.

Objective: The primary objective of this project is to develop and validate a multi-agent drone formation control framework that:

- Enables autonomous UAVs to self-organize from arbitrary initial positions into predefined geometric formations using Artificial Potential Field (APF) methods.
- Utilizes the Detective Deviative Pursuit (DDP) strategy for efficient leader-follower-based formation tracking and task execution with minimal computational cost.
- Demonstrates the effectiveness of this hybrid control strategy through simulation and physical experiments using the Crazyflie 2.1 nano drone platform.
- Ensures inter-agent collision avoidance, accurate trajectory tracking, and formation stability in constrained environments.
- Provides a scalable control architecture with the potential for extension to outdoor and obstacle-rich deployments.

1.7 LAYOUT

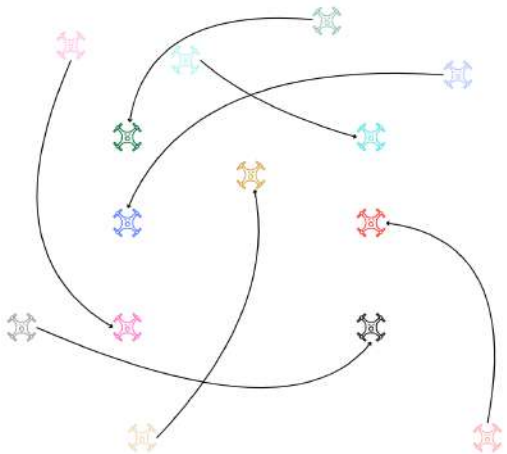
Chapter 2 2 of this report presents the problem description and introduces the background theory related to Detective Derivative Pursuit (DDP) and the Artificial Potential Field (APF) based guidance law. Chapter 3 3 then details the hardware implementation of these control strategies using a nano quadrotor platform known as Crazyflie. The final chapter 4 concludes the report and outlines directions for future work.

CHAPTER 2

PROBLEM DESCRIPTION AND BACKGROUND

2.1 PROBLEM DESCRIPTION

Coordinating a swarm of autonomous aerial agents within a constrained environment presents significant challenges, particularly in maintaining formation, avoiding obstacles, and tracking dynamic targets. These capabilities are critical for real-world applications such as surveillance, mapping, and collaborative inspection. Achieving reliable multi-agent coordination demands robust control architectures, efficient data processing, and adaptive strategies to handle moving targets and changing conditions.



This project develops a drone formation control system based on the Crazyflie 2.1 platform, leveraging ROS and the Crazyswarm package for real-time communication and control. The system employs Artificial Potential Field (APF) methods to enable collision avoidance. Additionally, a variation of the leader-follower approach—Detective Derivative Pursuit (DDP)—is implemented to manage formation control and task execution. DDP allows agents to adjust their trajectories by detecting relative motion and inferring the paths of moving or virtual targets, facilitating effective pursuit behavior. The framework is validated through both simulation and experimental flight tests, with drones successfully forming and maneuvering through letter-shaped formations in a shared workspace.

2.1.1 Mission

Due to the limited number of drones, a relatively simple yet visually demonstrative mission was designed for this project. The objective of the mission is to sequentially form the letters **'I'**, **'I'**,**'T'** and **'M'** in a horizontal plane using a team of seven crazyflies.

The sequence of the mission is outlined as follows:

1. **Takeoff**: All seven drones take off from their designated initial positions on the ground.

2. **First Formation – 'I'**: Three drones form a vertical line representing the letter 'I', while the other four remain in standby, hovering nearby.

3. **Translation**: The 'I' formation translates laterally across the vertical plane to the opposite side of the workspace, maintaining the formation throughout the motion.

4. **Deformation**: Upon reaching the far end, the drones in the 'I' formation deform and return.

5. **Second Formation – 'I'**: Same set of three drones reform the letter 'I', move laterally to the other end and return.

6. **Third and Fourth Formation – 'T'**: Now five drones coordinate to form the letter 'T' and perform the same movement followed by letter 'M' with seven drones.

7. **Landing**: After the 'M' formation reached the other end, all drones return and land simultaneously.

In this mission, Artificial Potential field based guidance law is used to form the letter configurations and detective deviated pursuit is used to move the formation laterally.

2.2 BACKGROUND

2.2.1 Detective Deviative Pursuit

The **Detective Deviated Pursuit (DDP)** guidance law is designed to allow a follower agent to maintain a constant distance from a leader while deviating its velocity vector by a fixed angular offset relative to the line of sight (LOS). Unlike traditional guidance problems where interception is the goal, DDP emphasizes maintaining formation by enforcing a fixed range.

2.2.1.1 DDP Geometry for One Follower Configuration

Key variables are shown using the one follower configuration.

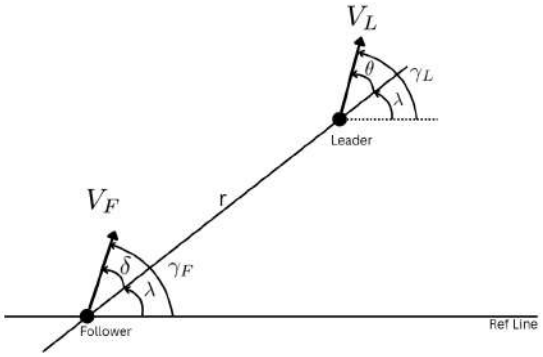


Figure 2.1: One Follower Cofiguration

- L: Leader agent
- F: Follower agent
- V_L, V_F: Linear speeds of leader and follower
- γ_L, γ_F: Path angles of leader and follower
- r: Distance between leader and follower
- λ: Line-of-sight (LOS) angle
- θ: Angle between leader's velocity and LOS
- δ: Constant deviation angle between follower velocity and LOS

2.2.1.1.1 DDP Conditions

1. **Detective Condition:** Follower maintains a constant range to the leader:

$$\dot{r} = 0 \Rightarrow V_L \cos\theta - V_F \cos\delta = 0 \tag{2.1}$$

2. **Deviation Condition:** Follower velocity deviates from LOS by constant δ, leading to:

$$\dot{\lambda} = \frac{1}{r}(V_L \sin\theta - V_F \sin\delta) \tag{2.2}$$

2.2.1.1.2 Path Angle Relationships

$$\gamma_L = \theta + \lambda \tag{2.3}$$

$$\gamma_F = \delta + \lambda = \delta + \gamma_L - \theta \tag{2.4}$$

2.2.1.1.3 Remarks

- If $\dot{\gamma}_L = 0$ (straight path), then $\theta = \delta$.
- If $\dot{\gamma}_L = \omega = \text{const}$ (circular path), additional angular velocity dynamics are considered.

2.2.1.2 DDP for Two Follower Configuration

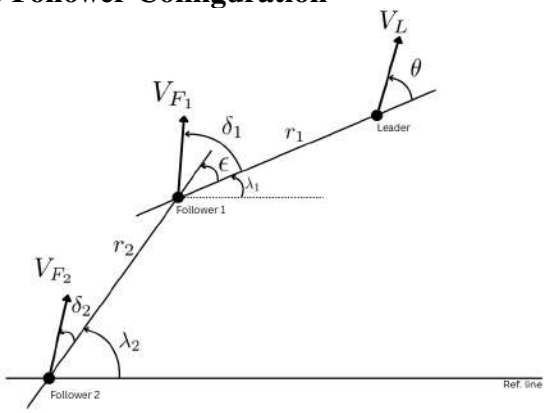


Figure 2.2: Two Follower Configuration

- L: Leader agent
- F_1,F_2: Follower 1, Follower 2
- V_L, V_{F_1}, V_{F_2}: Linear speeds of leader and followers
- r_1: Distance between leader and follower 1
- r_2: Distance between follower 1 and follower 2
- λ_1: Line-of-sight (LOS) angle between leader and follower 1
- λ_2: Line-of-sight (LOS) angle between follower 1 and follower 2

- θ: Angle between leader's velocity and LOS
- δ_1,δ_2: Constant deviation angle between follower velocity and LOS
- ϵ: Inter-LOS angle

2.2.1.2.1 DDP Conditions

- **Range Rate Equations(Detective Condition)**

$$\dot{r}_1 = V_L \cos\theta - V_{F1} \cos\delta_1 \tag{19}$$
$$\dot{r}_2 = V_{F1} \cos(\delta_1 - \epsilon) - V_{F2} \cos\delta_2 \tag{20}$$

- **LOS Angular Rate Equations(Deviation Condition)**

$$\dot{\lambda}_1 = \frac{1}{r_1} (V_L \sin\theta - V_{F1} \sin\delta_1) \tag{21}$$
$$\dot{\lambda}_2 = \frac{1}{r_2} (V_{F1} \sin(\delta_1 - \epsilon) - V_{F2} \sin\delta_2) \tag{2.5}$$

- **Velocity Conditions - Equating range rate to 0**

$$V_{F1} = \frac{V_L \cos\theta}{\cos\delta_1} \tag{23}$$
$$V_{F2} = \frac{V_L \cos\theta \cos(\delta_1 - \epsilon)}{\cos\delta_1 \cos\delta_2} \tag{2.6}$$

- **Inter-LOS Angle**

$$\epsilon = \lambda_2 - \lambda_1 \tag{2.7}$$

2.2.1.3 Generalized DDP Geometry for N-Followers

To extend the Detective Deviated Pursuit (DDP) control framework to a multi-agent formation, we analyze the local interaction geometry involving follower N, follower $N-1$, and follower $N-2$. Based on this relative configuration, the range rate and line-of-sight (LOS) angular rate for follower N can be expressed as:

$$\dot{r}_N = V_{F_{N-1}} \cos(\delta_{N-1} - \epsilon_{N-1}) - V_{F_N} \cos(\delta_N) \tag{2.8}$$

$$\dot{\lambda}_N = \frac{1}{r_N} \left[V_{F_{N-1}} \sin(\delta_{N-1} - \epsilon_{N-1}) - V_{F_N} \sin(\delta_N) \right] \tag{2.9}$$

Where:

$$V_{F_0} = V_L \tag{2.10}$$

$$\delta_0 = \theta \tag{2.11}$$

$$\epsilon_0 = 0 \tag{2.12}$$

$$\epsilon_N = \lambda_{N+1} - \lambda_N \tag{2.13}$$

To satisfy the **detective condition** (i.e., constant distance), the range rate must be zero:

$$\dot{r}_N = 0 \tag{2.14}$$

Substituting this into equation (1), we obtain the required velocity for follower N as:

$$V_{F_N} = V_{F_{N-1}} \frac{\cos(\delta_{N-1} - \epsilon_{N-1})}{\cos(\delta_N)} \tag{2.15}$$

This recursive formulation allows for scalable control of large formations using local interactions.

2.2.1.4 Validation using MATLAB

Simulations of the Detective Deviative Pursuit (DDP) algorithm were conducted for configurations involving one, two, and three follower drones. The velocity of the leader drone was maintained at 1 m/s in all scenarios, while the angular velocity was set to $3^\circ/\text{s}$, 0.1 rad/s, and 0.06 rad/s, respectively, for each configuration. The initial positions of the drones and the direction of the leader's velocity vector are depicted in the simulation plots. As illustrated in Figures 2.3, 2.4, and 2.5, the inter-agent distances remain

consistent throughout the simulation runs, validating the stability and effectiveness of the DDP-based formation control strategy under varying motion dynamics.

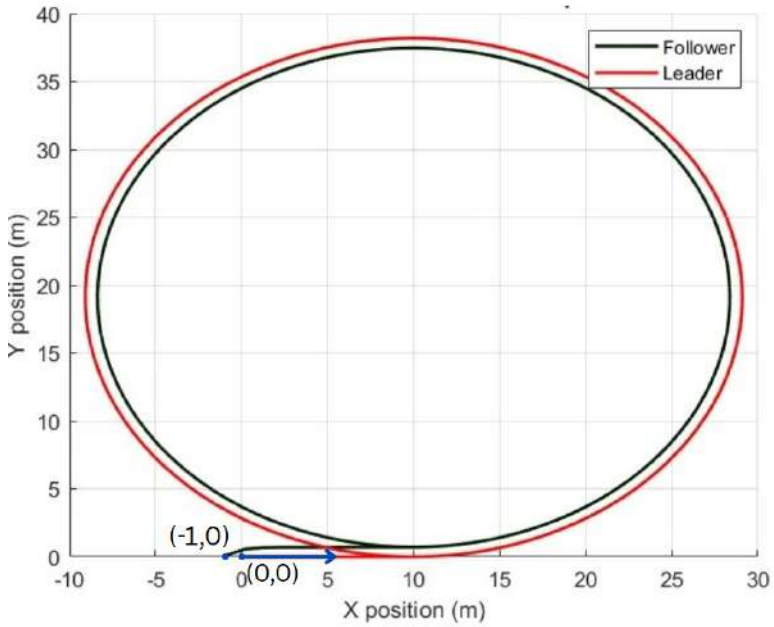


Figure 2.3

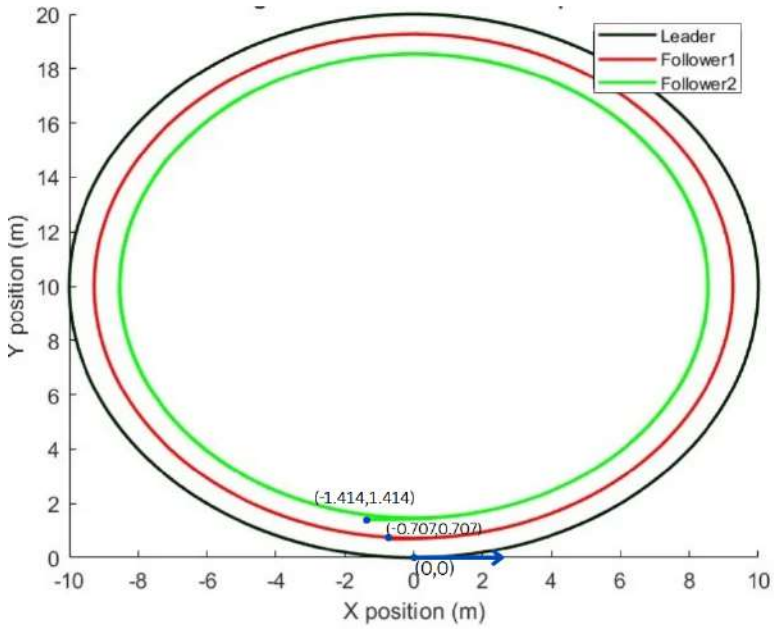


Figure 2.4

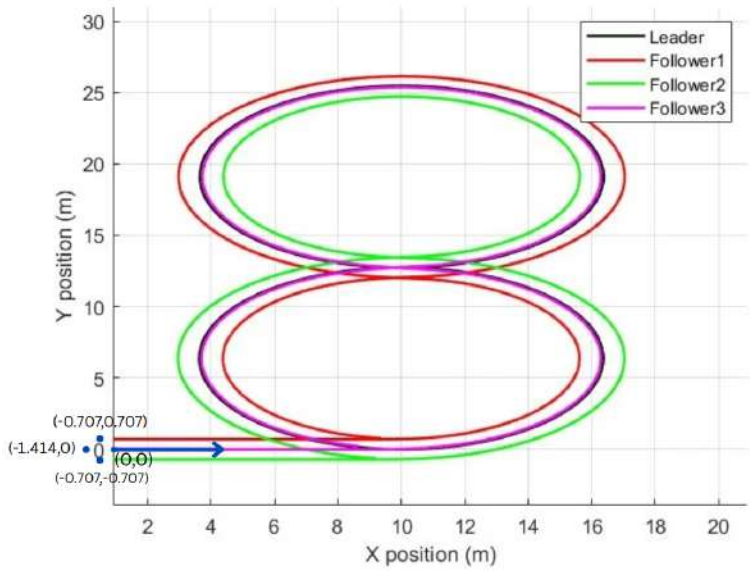


Figure 2.5

2.2.2 Artificial Potential Field Based Avoidance

The **Artificial Potential Field**(APF) based guidance law is designed to allow multiple agents to navigate toward their respective targets while avoiding collisions in real time. Unlike global path planning techniques, APF utilizes a local, reactive control policy that dynamically adjusts each agent's velocity based on the proximity of nearby agents. Emergent collision-free behavior is obtained from simple, local interactions.

Consider an agent A_i located at position $\mathbf{x}_i$, navigating toward its target position $\mathbf{x}_i^{\text{target}}$. If another agent A_j enters the collision radius threshold around A_i, the APF based law is employed to compute a resultant velocity that enables A_i to avoid potential collisions while maintaining progress toward its assigned goal.

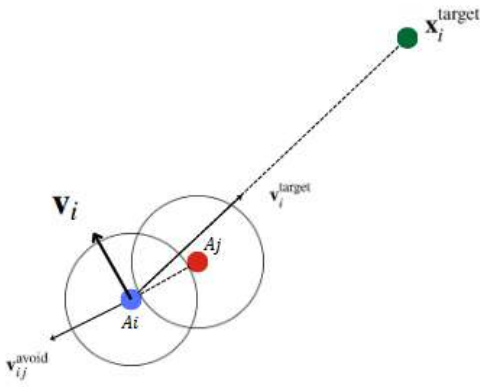


Figure 2.6: APF based Avoidance is used to guide A_i to the target position $\mathbf{x}_i^{\text{target}}$ while avoiding A_j

- A_i: The i-th agent (current agent under consideration)
- A_j: A neighboring agent that could potentially collide with A_i
- $\mathbf{x}_i, \mathbf{x}_j$: Current positions of agents i and j
- $\mathbf{x}_i^{\text{target}}$: Target position of agent i
- $\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i$: Relative position vector
- r_c: Collision threshold radius
- $v_{\max}$: Maximum allowed speed for each agent

2.2.2.1 APF Conditions

1. **Attraction to Target:** Each agent seeks to move toward its target with velocity:

$$\mathbf{v}_i^{\text{target}} = \begin{cases} \dfrac{\mathbf{x}_i^{\text{target}} - \mathbf{x}_i}{\|\mathbf{x}_i^{\text{target}} - \mathbf{x}_i\| + \varepsilon} \cdot v_{\max}, & \text{if } \|\mathbf{x}_i^{\text{target}} - \mathbf{x}_i\| > \epsilon_t \\ \mathbf{0}, & \text{otherwise} \end{cases} \tag{2.16}$$

   where ε is a small constant to avoid division by zero and ϵ_t is a threshold for goal tolerance.

2. **Repulsive Avoidance:** If a neighboring agent j is within a critical collision radius

r_c, a repulsive velocity is added:

$$\mathbf{v}_{ij}^{\text{avoid}} = -\frac{\mathbf{r}_{ij}}{\|\mathbf{r}_{ij}\| + \varepsilon} \cdot v_{\max}, \quad \text{if } \|\mathbf{r}_{ij}\| < r_c \tag{2.17}$$

The total avoidance-adjusted velocity is:

$$\mathbf{v}_i = \mathbf{v}_i^{\text{target}} + \sum_{j \neq i} \mathbf{v}_{ij}^{\text{avoid}} \tag{2.18}$$

3. **Speed Constraint:** The resultant velocity is clipped to the maximum allowable speed:

$$\text{if } \|\mathbf{v}_i\| > v_{\max}, \quad \mathbf{v}_i \leftarrow \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|} \cdot v_{\max} \tag{2.19}$$

2.2.2.2 Validation using Python

To validate the Artificial Potential Field (APF)-based guidance law, six agents were simulated within a $2 \times 2\,\text{m}^2$ workspace using Python. Each agent was assigned random initial and target positions. The maximum velocity for each agent was limited to 0.1 m/s, and the collision threshold radius—defined as the minimum allowable distance between any two agents—was set to 0.28 m. The simulation results are illustrated in Figure 2.7.

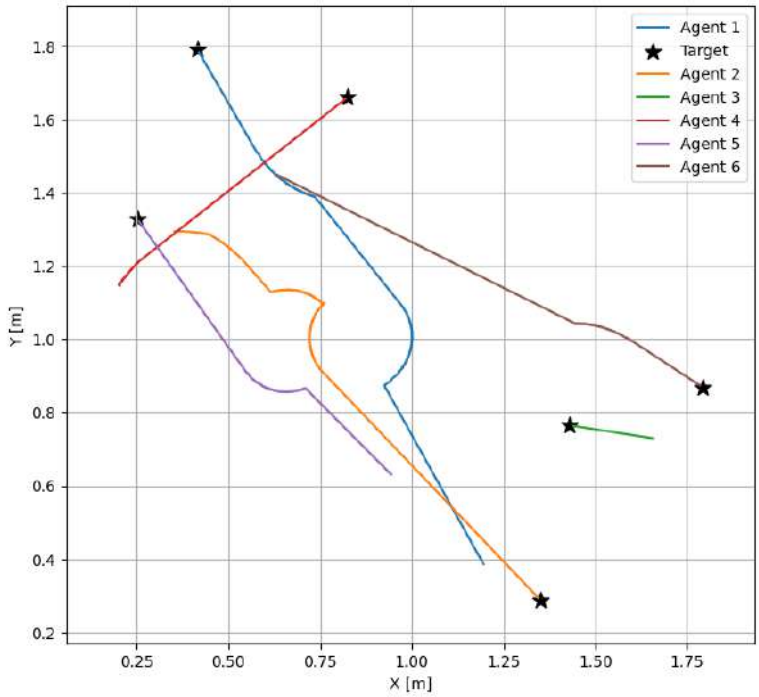


Figure 2.7: Multi-Agent Navigation with Global Planner + APF based Avoidance. Each agent navigates directly toward its target along a straight-line path when no nearby agents are detected. However, if another agent is in close proximity, the agent dynamically adjusts its trajectory to plan a path around the nearby obstacle to avoid collision.

CHAPTER 3

IMPLEMENTATION OF THE MISSION USING MULTIPLE CRAZYFLIES

3.1 SETUP

Crazyflie is an open-source nano quadrotor platform with compact form factor developed by Bitcraze AB, designed primarily for research, education, and prototyping in the field of aerial robotics. widely adopted in research due to its modular architecture, which allows for the integration of advanced features through the addition of specialized hardware modules known as decks. Some commonly used decks include the Lighthouse Deck for precise positioning using external tracking systems, the Flow Deck for optical flow-based velocity estimation, and the Multiranger Deck for obstacle detection and ranging using multiple time-of-flight sensors. In this project, the Lighthouse Deck is used to provide accurate positional information, which serves as input for the formation flight control algorithm.

Figure 3.1: Crazyflie

Figure 3.2: Lighthouse Deck

The Lighthouse Deck requires the presence of Lighthouse Base Stations, infrared-emitting devices, to operate. These base stations emit sweeping IR laser beams that are detected by photodiodes on the deck, allowing the Crazyflie to estimate its position with high accuracy in a 3D space. For optimal performance, at least two base stations are required. For this project, four base stations were used.

Figure 3.3: Lighthouse Base Station

3.2 CRAZYSWARM

Crazyswarm is a large-scale swarm robotics platform designed for the control and coordination of multiple Crazyflies. It provides an integrated software stack that enables precise indoor flight of dozens of drones simultaneously. The system is built on top of the Crazyflie firmware and leverages motion capture systems (such as VICON or OptiTrack) or local positioning (such as Lighthouse) for real-time position tracking.

The core of Crazyswarm is its ROS-based (Robot Operating System) architecture, which allows easy integration with custom code, high-level planning algorithms, and external sensors. The software supports trajectory planning, real-time command broadcasting, and synchronized multi-agent control. It also includes a Python API, which makes it convenient to define and execute complex swarm behaviors programmatically.

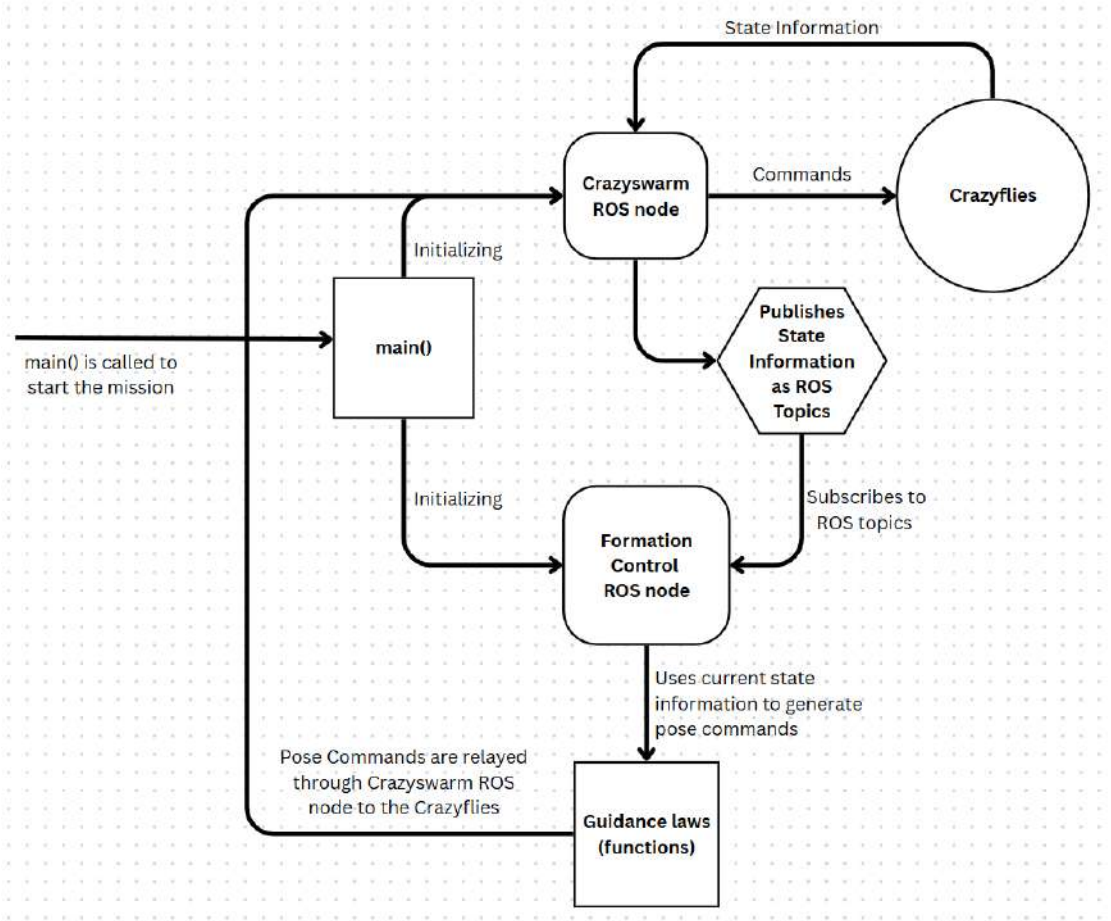
In this project, Crazyswarm was used to handle low-level flight control, state estimation, and position broadcasting, while the high-level navigation and swarm coordination algorithms were implemented separately. The use of Crazyswarm allowed rapid prototyping and reliable execution of multi-agent flight experiments in a controlled indoor environment.

3.3 ROBOT OPERATING SYSTEM

The Robot Operating System (ROS) served as the middleware framework for the guidance system implemented in this project. ROS provided the communication infrastructure, tools, and libraries necessary to manage multiple Crazyflie drones through the Crazyswarm framework.ROS nodes were used to handle real-time position updates from the local positioning system, issue control commands to each Crazyflies, and implement the high-level guidance logic. By leveraging ROS topics and services, commands could be broadcast efficiently to the swarm while maintaining flexibility for individual drone behavior if needed.

3.4 GUIDANCE CODE

Code structure



The main function, given in 3.1 initializes the `Crazyswarm` ROS node and launches a multi-threaded executor for the `FormationControl` node. One thread of this executor continuously spins in the background to update and maintain the real-time position information of all drones in the swarm.

```
def main()
    swarm=Crazyswarm()
    executor=MultiThreadedExecutor()
    control=FormationControl(swarm)
    executor.add_node(control)
    spin_thread = threading.Thread(target=background_spin, args=(
```

```
        ↪ executor, control), daemon=True)
    spin_thread.start()

def background_spin(executor, node):
    executor.spin()
```

Listing 3.1: The Crazyswarm() class is imported from the crazyflie_py library within the Crazyswarm package and is initialized in the main() function. This initialization sets up the Crazyswarm ROS node, which establishes communication with all connected Crazyflie drones. Additionally, a FormationControl object is instantiated and added as a node to the multi-threaded ROS executor, enabling concurrent execution and continuous handling of control logic and state updates.

Initializing the Crazyswarm node enables communication with the Crazyflie drones and begins publishing their state information as ROS topics. Subsequently, initializing the `FormationControl` object sets up subscriptions to these topics, allowing it to access the necessary state data for control and coordination. The relevant code implementation is provided in A.1.

Basic functions such as takeoff and landing are executed using the `Crazyswarm.allcfs` interface, as demonstrated in A.2.

The remaining part of the `main` function, which sequentially calls the necessary functions to execute all required formations and movements as part of the mission flow, is presented below.

```
    try:
        control.timeHelper.sleep(5.0)
        control.takeoff()
        control.timeHelper.sleep(5.0)

```

```
        control.formI()
        control.timeHelper.sleep(5.0)
        control.moveI()
        control.timeHelper.sleep(5.0)
        control.returnI()
        control.timeHelper.sleep(5.0)

        control.formI()
        control.timeHelper.sleep(5.0)
        control.moveI()
        control.timeHelper.sleep(5.0)
        control.returnI()
        control.timeHelper.sleep(5.0)

        control.formT()
        control.timeHelper.sleep(5.0)
        control.moveT()
        control.timeHelper.sleep(5.0)
        control.returnT()
        control.timeHelper.sleep(5.0)

        control.formM()
        control.timeHelper.sleep(5.0)
        control.moveM()
        control.timeHelper.sleep(5.0)
        control.returnM()
        control.timeHelper.sleep(5.0)
        control.land()

    except KeyboardInterrupt:
        pass
    finally:
        control.destroy_node()
        rclpy.shutdown()
```

Listing 3.2: The remaining part of the main function is shown. This part executes a sequence of coordinated drone maneuvers. All drones take off simultaneously using the `takeoff` function. Subsequently, the formation routines `formI`, `moveI`, and `returnI` are called sequentially and repeated twice to form two instances of the letter 'I'. The drones then form the letter 'T' using `formT`, perform a movement using `moveT`, and return using `returnT`. A similar sequence is executed for the letter 'M' using corresponding functions. Finally, all drones are landed simultaneously using the `land` function.

The function implementing the Artificial Potential Field (APF)-based guidance law, described in 2.2.2, is used to achieve formations resembling the letters "I", "T", and "M". The corresponding implementations are presented in A.3, A.4, and A.5, respectively. These implementations follow a similar structure, differing primarily in the target positions assigned to each UAV.

The `goTo` command from the `Crazyswarm` library is used to broadcast the calculated pose commands to the Crazyflies. This function sends position and orientation setpoints to each drone, enabling them to move towards their desired target locations as computed by the guidance algorithm.

The `moveI` (A.6), `moveT` (A.7), and `moveM` (A.8) functions implement the Detective Deviated Pursuit (DDP) guidance law, as described in 2.2.1, to control the motion of drones forming the letters "I", "T", and "M", respectively. These functions utilize the `solve_ivp` method to numerically solve the system of differential equations governing the agents' dynamics. The equations are defined in the `kinematics_I`, `kinematics_T`, and `kinematics_M` functions, which encode the kinematic models and DDP-based control laws for each formation. In the case of the 'I' and 'M' formations, the leader drone is positioned in line with one or more follower drones. To ensure proper calculation within

the formation control equations, a positive offset is applied to the leader's position in the direction of its velocity. This offset allows for meaningful relative position computations between the leader and its followers, which would otherwise be undefined.

After completing the lateral motion, the drone formations are collapsed and redirected to the initial corner position in preparation for the next formation. This return maneuver is implemented using the APF-based guidance law. The corresponding code is provided in A.5.

3.5 IMPLEMENTATION RESULTS

Snapshots of the implementation are provided below.

Full video demonstration: link

Note that the formations in the figures may appear distorted due to the fisheye lens used to capture a wide field of view.

Figure 3.4: Formation 'I' using 3 drones and 4 drones hovering nearby

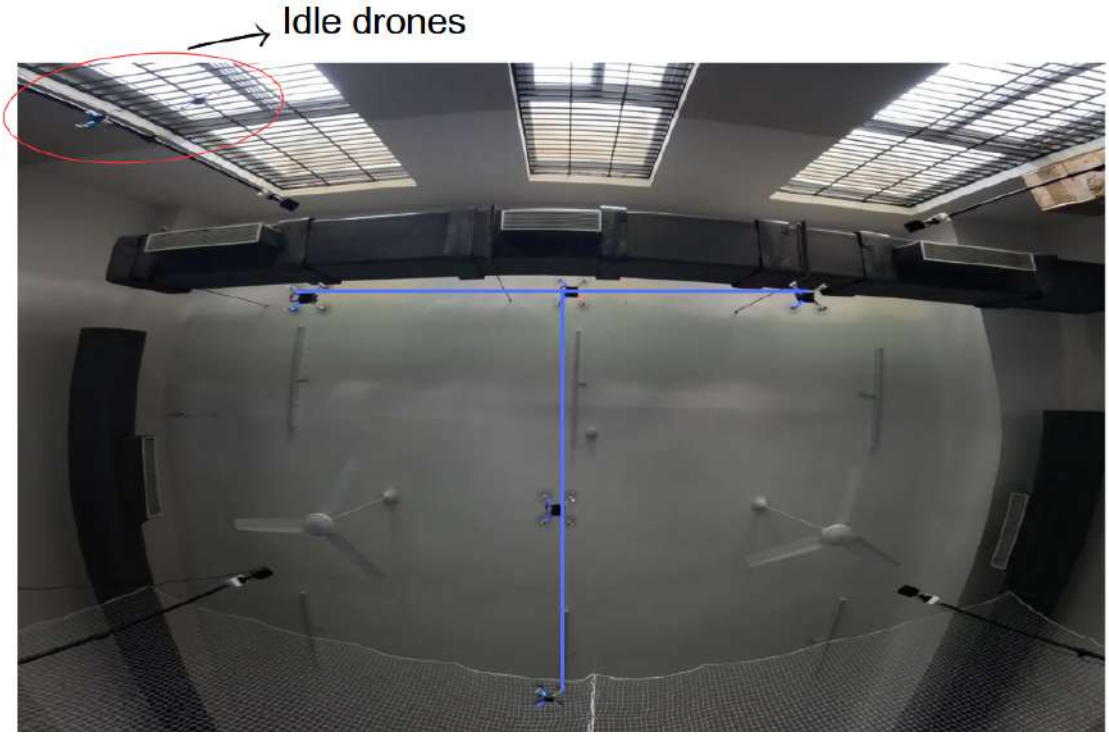


Figure 3.5: Formation ’T’ using 5 drones and 2 drones hovering nearby

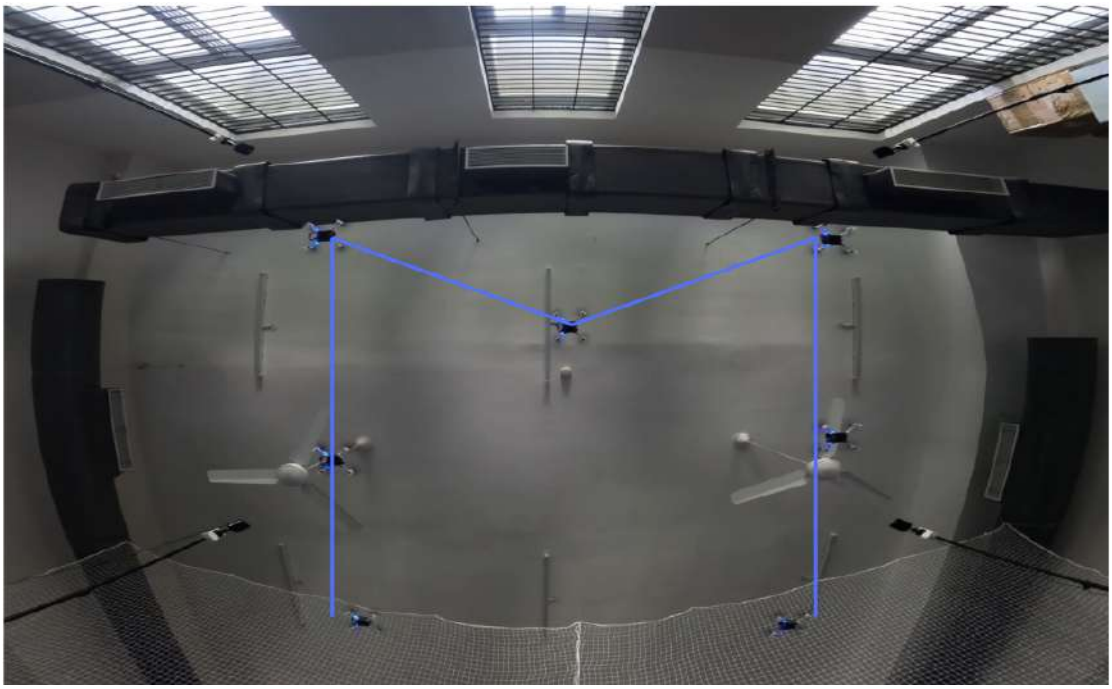


Figure 3.6: Formation ’M’ using 7 drones

CHAPTER 4

CONCLUSION AND FUTURE WORK

This project successfully demonstrates the implementation of a real-time multi-agent drone formation control system using the Crazyflie platform and the Crazyswarm ROS framework. By integrating Artificial Potential Field (APF) methods for collision avoidance and Detective Derivative Pursuit (DDP) for efficient pursuit and formation control, the system achieves reliable coordination among multiple quadrotor drones operating within a constrained indoor environment.

Both simulation and experimental flight tests validate the system's effectiveness, as the drones accurately form and navigate through predefined configurations such as the letters 'I', 'T', and 'M'. The approach offers a balanced trade-off between computational efficiency and robustness, particularly through the use of DDP to reduce the computational burden typically associated with centralized control strategies.

Overall, this work lays a foundation for scalable and adaptive swarm robotics applications. While the current implementation demonstrates reliable performance in controlled indoor environments, several extensions can further enhance the capability, scalability, and robustness of the system.

- **Leader-Independent DDP:** The current Detective Deviative Pursuit (DDP) strategy depends on a designated leader. A decentralized, virtual leader or leader-independent variant would improve resilience to single-point failures and enable more flexible swarm behaviors.

- **Obstacle-Rich Environments:** Integrating the system with real-time obstacle detection and avoidance mechanisms would allow operation in complex, unstructured, or dynamic environments—beyond the current assumption of an open workspace.

- **Outdoor Deployment with GPS:** Extending the system to support GPS-based localization will allow outdoor deployment, making it viable for real-world

applications such as search and rescue, precision agriculture, and infrastructure inspection.

- **Heterogeneous Agent Coordination:** Future work may also explore the use of heterogeneous agents—combining drones with different sensing or actuation capabilities—to perform collaborative tasks that cannot be handled by identical platforms alone.

- **Energy-Aware Path Planning:** Incorporating energy constraints into formation planning and movement decisions can improve efficiency and prolong mission duration, especially in large-scale deployments.

APPENDIX A

SOURCE CODES

A.1 INITIALIZING

```
class FormationControl(Node):


    def __init__(self,swarm):
        super().__init__('FormationControl')
        self.swarm=swarm
        self.timeHelper=self.swarm.timeHelper
        self.initialize_subscribers()

    def initialize_subscribers(self):
        self.pose_subscribers={}
        self.positions={}
        self.orientations={}
        self.thetas={}
        self.alphas={}
        self.deltas={}

        for cf in self.swarm.allcfs.crazyfliesByName:
            self.pose_subscribers[cf]=self.pose_subscriber_fun(cf)

    def pose_subscriber_fun(self,name):
        self.subscription = self.create_subscription(
            PoseStamped,
            f"/{name}/pose",
            lambda msg: self.pose_callback(msg,name),
            10,
            callback_group=self.pose_callback_group)
        return self.subscription
```

```
28
29      def pose_callback(self, msg,name):
30          self.positions[name]=[msg.pose.position.x,msg.pose.position.y
                ↪ ,msg.pose.position.z]
31          self.orientations[name]=self.quaternion_to_euler(msg.pose.
                ↪ orientation)
32          self.calculateAlphasAndThetas(name)
33
34      def quaternion_to_euler(self,orientation):
35          q = [orientation.x, orientation.y, orientation.z, orientation
                ↪ .w]
36          r = R.from_quat(q)
37          euler = r.as_euler('xyz', degrees=False)
38          return euler
```

Listing A.1: The pose_subscriber_fun() function initializes the necessary ROS subscribers to receive pose data from each agent. The incoming data is processed by the pose_callback() function, which converts it into a usable format suitable for subsequent stages of the mission.

A.2 TAKEOFF & LAND

```
1       def takeoff(self):
2           self.swarm.allcfs.arm()
3           self.swarm.allcfs.takeoff(targetHeight=1.0, duration=3.0)
4
5       def land(self):
6           self.swarm.allcfs.land(targetHeight=0.05, duration=3.0)
```

Listing A.2: Takeoff and Land

A.3 FORM

```
    def formI(self):


        X=np.array([
            [self.positions[cf][0],self.positions[cf][1],1.0] for cf
                ↪ in self.swarm.allcfs.crazyfliesByName
        ])

        n_agents = 7
        dim = 3
        dt = 0.05
        max_speed = 0.3
        collision_radius = 0.28
        goal_tolerance = 0.01

        V = np.zeros_like(X)
        history = [X.copy()]

        def all_reached(X, X_target, tol=goal_tolerance):
            return np.all(np.linalg.norm(X - X_target, axis=1) < tol)

        X_target = np.array([
            [0.0, -1.2, 1.0],
            [0.0, -0.6, 1.0],
            [0.0, -1.8 , 1.0],
            [0.2,0.2,1.0],
            [-0.1,0.2,1.0],
            [-0.4,0.2,1.0],
            [-0.7,0.2,1.0]
        ])

        def compute_vo_velocity(X, V, i, X_target):
            to_target = X_target[i] - X[i]
```

```
            dist = np.linalg.norm(to_target)
            desired = to_target / (dist + 1e-6) * max_speed if dist >
                ↪  1e-3 else np.zeros(3)

            for j in range(n_agents):
                if i != j:
                    rel_pos = X[j] - X[i]
                    d = np.linalg.norm(rel_pos)
                    if d < collision_radius:
                        avoidance = -rel_pos / (d + 1e-6) * max_speed
                        desired += avoidance

            speed = np.linalg.norm(desired)
            if speed > max_speed:
                desired = desired / speed * max_speed
            return desired

        max_iters = 2000
        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1


        history=history[::3]
        cfs={f'cf{i}': i-1 for i in range(1,8)}
```

```
        print(cfs)

        step=len(history)

        for i in range(step):
            target=history[i]
            t=5.0/step
            # print(target)
            for cfName in self.swarm.allcfs.crazyfliesByName:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
                print(f'{cfName}:{target[cfs[cfName]]}')
            self.timeHelper.sleep(t)
```

Listing A.3: Formation of 'I'

```
    def formT(self):
        X=np.array([
            [self.positions[cf][0],self.positions[cf][1],1.0] for cf
                ↪ in self.swarm.allcfs.crazyfliesByName
        ])

        n_agents = 7
        dim = 3
        dt = 0.05
        max_speed = 0.3
        collision_radius = 0.28
        goal_tolerance = 0.01

        V = np.zeros_like(X)
        history = [X.copy()]

        def all_reached(X, X_target, tol=goal_tolerance):
            return np.all(np.linalg.norm(X - X_target, axis=1) < tol)
```

```
18
19          X_target = np.array([
20              [0.6, -0.6, 1.0],
21              [0.0, -0.6, 1.0],
22              [0.0, -1.2 , 1.0],
23              [0.0, -1.8 , 1.0],
24              [-0.6, -0.6 , 1.0],
25              [-0.4,0.2,1.0],
26              [-0.7,0.2,1.0]
27          ])
28
29          def compute_vo_velocity(X, V, i, X_target):
30              to_target = X_target[i] - X[i]
31              dist = np.linalg.norm(to_target)
32              desired = to_target / (dist + 1e-6) * max_speed if dist >
                    ↪  1e-3 else np.zeros(3)
33
34              for j in range(n_agents):
35                  if i != j:
36                      rel_pos = X[j] - X[i]
37                      d = np.linalg.norm(rel_pos)
38                      if d < collision_radius:
39                          avoidance = -rel_pos / (d + 1e-6) * max_speed
40                          desired += avoidance
41
42              speed = np.linalg.norm(desired)
43              if speed > max_speed:
44                  desired = desired / speed * max_speed
45              return desired
46
47          max_iters = 2000
48          step = 0
49
50          while not all_reached(X, X_target) and step < max_iters:
```

```
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1


        history = np.array(history)


        history=history[::3]
        cfs={f'cf{i}': i-1 for i in range(1,8)}
        print(cfs)

        step=len(history)

        for i in range(step):
            target=history[i]
            t=5.0/step
            print(target)
            for cfName in self.swarm.allcfs.crazyfliesByName:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
                print(f'{cfName}:{target[cfs[cfName]]}')
            self.timeHelper.sleep(t)
```

Listing A.4: Formation of 'T'

```
    def formM(self):
        X=np.array([
```

```
            [self.positions[cf][0],self.positions[cf][1],1.0] for cf
                ↪ in self.swarm.allcfs.crazyfliesByName
        ])

        n_agents = 7
        dim = 3
        dt = 0.05
        max_speed = 0.3
        collision_radius = 0.28
        goal_tolerance = 0.01

        V = np.zeros_like(X)
        history = [X.copy()]

        def all_reached(X, X_target, tol=goal_tolerance):
            return np.all(np.linalg.norm(X - X_target, axis=1) < tol)

        X_target = np.array([
            [0.6, -1.2, 1.0],
            [0.6, -0.6, 1.0],
            [0.6, -1.8 , 1.0],
            [0.0, -0.9 , 1.0],
            [-0.6, -0.6 , 1.0],
            [-0.6, -1.2 , 1.0],
            [-0.6,-1.8,1.0]
        ])

        def compute_vo_velocity(X, V, i, X_target):
            to_target = X_target[i] - X[i]
            dist = np.linalg.norm(to_target)
            desired = to_target / (dist + 1e-6) * max_speed if dist >
                ↪  1e-3 else np.zeros(3)

            for j in range(n_agents):
```

```
                if i != j:
                    rel_pos = X[j] - X[i]
                    d = np.linalg.norm(rel_pos)
                    if d < collision_radius:
                        avoidance = -rel_pos / (d + 1e-6) * max_speed
                        desired += avoidance

            speed = np.linalg.norm(desired)
            if speed > max_speed:
                desired = desired / speed * max_speed
            return desired

        max_iters = 2000
        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1


        history = np.array(history)


        history=history[::3]
        cfs={f'cf{i}': i-1 for i in range(1,8)}
        print(cfs)

```

```
        step=len(history)

        for i in range(step):
            target=history[i]
            t=5.0/step
            print(target)
            for cfName in self.swarm.allcfs.crazyfliesByName:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
                print(f'{cfName}:{target[cfs[cfName]]}')
            self.timeHelper.sleep(t)
        print(step)
```

Listing A.5: Formation of 'M'

```
    def moveI(self):
        self.VL = 0.3
        self.R = 0.6*np.sqrt(2)
        self.delta1 = 45 * np.pi / 180
        self.delta2 = -45 * np.pi / 180

        self.y=[self.thetas['cf2'],self.thetas['cf3'],self.alphas['
            ↪ cf1'],self.alphas['cf2'],self.alphas['cf3'],self.
            ↪ positions['cf1'][0]+self.R*np.cos(self.delta1),self.
            ↪ positions['cf1'][1],self.positions['cf2'][0],self.
            ↪ positions['cf2'][1],self.positions['cf3'][0],self.
            ↪ positions['cf3'][1]]

        t_span = (0,6)
        x_pts = np.linspace(0, 6, 101)
        sol = solve_ivp(self.kinematics_I, t_span, self.y,t_eval=
            ↪ x_pts)
```

```
12
13          num=len(sol.y[0])
14          for n in range(num):
15              column = [row[n] for row in sol.y]
16              # print(column)
17              # print('yes')
18              pos={}
19
20              pos['cf1']=[column[5]-self.R*np.cos(self.delta1),column
                    ↪ [6],1.0]
21              pos['cf2']=[column[7],column[8],1.0]
22              pos['cf3']=[column[9],column[10],1.0]
23
24              cfs={f'cf{i}': i-1 for i in range(1,4)}
25              for cfName in cfs:
26                  cf=self.swarm.allcfs.crazyfliesByName[cfName]
27                  cf.goTo(pos[cfName],0,0.8)
28              self.swarm.timeHelper.sleep(0.06)
29
30      def kinematics_I(self,t,y):
31          self.R = 0.6*np.sqrt(2)
32          self.delta1 = 45 * np.pi / 180
33          self.delta2 = -45 * np.pi / 180
34          theta1, theta2 = -self.delta1, -self.delta2
35          alpha_L, alpha_f1, alpha_f2 = y[2],y[3], y[4]
36
37          xl,yl=y[5],y[6]
38          x1,y1=y[7],y[8]
39          x2,y2=y[9],y[10]
40
41          Vf1 = self.VL * np.cos(alpha_L - theta1) / np.cos(self.delta1
                ↪ )
42          Vf2 = self.VL * np.cos(alpha_L - theta2) / np.cos(self.delta2
                ↪ )
```

```
43
44
45          dydt = np.zeros_like(y)
46
47          dydt[0] = (self.VL * np.sin(alpha_L - theta1 - self.delta1))
                ↪ / (self.R * np.cos(self.delta1))
48          dydt[1] = (self.VL * np.sin(alpha_L - theta2 - self.delta2))
                ↪ / (self.R * np.cos(self.delta2))
49
50          dydt[2] = 0
51          dydt[3] = dydt[0]
52          dydt[4] = dydt[1]
53
54          dydt[5] = self.VL * np.cos(alpha_L)
55          dydt[6] = self.VL * np.sin(alpha_L)
56          dydt[7] = Vf1 * np.cos(alpha_f1)
57          dydt[8] = Vf1 * np.sin(alpha_f1)
58          dydt[9] = Vf2 * np.cos(alpha_f2)
59          dydt[10] = Vf2 * np.sin(alpha_f2)
60
61          return dydt
```

Listing A.6: Movement by 'I' using DDP

```
1       def moveT(self):
2           self.VL = 0.3
3           self.y=[self.thetas['cf2'],self.thetas['cf3'],self.thetas['
                ↪ cf4'],self.thetas['cf5'],self.alphas['cf1'],self.alphas
                ↪ ['cf2'],self.alphas['cf3'],self.alphas['cf4'],self.
                ↪ alphas['cf5'],self.positions['cf1'][0],self.positions['
                ↪ cf1'][1],self.positions['cf2'][0],self.positions['cf2'
                ↪ ][1],self.positions['cf3'][0],self.positions['cf3'][1],
                ↪ self.positions['cf4'][0],self.positions['cf4'][1],self.
                ↪ positions['cf5'][0],self.positions['cf5'][1]]
4
```

```
        t_span = (0,6)
        x_pts = np.linspace(0, 6, 101)
        sol = solve_ivp(self.kinematics_T, t_span, self.y,t_eval=
            ↪ x_pts)
        num=len(sol.y[0])
        for n in range(num):
            column = [row[n] for row in sol.y]

            pos={}
            pos['cf1']=[column[9],column[10],1.0]
            pos['cf2']=[column[11],column[12],1.0]
            pos['cf3']=[column[13],column[14],1.0]
            pos['cf4']=[column[15],column[16],1.0]
            pos['cf5']=[column[17],column[18],1.0]

            cfs={f'cf{i}': i-1 for i in range(1,6)}

            for cfName in cfs:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo(pos[cfName],0,0.8)
            self.swarm.timeHelper.sleep(0.06)

    def kinematics_T(self,t,y):

        R1=0.6
        R2=0.6*np.sqrt(2)
        R3=0.6*np.sqrt(5)
        R4=0.6

        self.delta1 = 0
        self.delta2 =-45*np.pi/180
        self.delta3=-1.10714872
        self.delta4=0

```

```
        theta1, theta2,theta3,theta4 = -self.delta1, -self.delta2,-
            ↪ self.delta3,-self.delta4
        alpha_L, alpha_f1, alpha_f2 ,alpha_f3,alpha_f4= y[4],y[5],y
            ↪ [6],y[7],y[8]

        xl,yl=y[9],y[10]
        x1,y1=y[11],y[12]
        x2,y2=y[13],y[14]
        x3,y3=y[15],y[16]
        x4,y4=y[17],y[18]

        epsilon=theta4-theta1
        Vf1 = self.VL * np.cos(alpha_L - theta1) / np.cos(self.delta1
            ↪ )
        Vf2 = self.VL * np.cos(alpha_L - theta2) / np.cos(self.delta2
            ↪ )
        Vf3 = self.VL * np.cos(alpha_L - theta3) / np.cos(self.delta3
            ↪ )
        Vf4=Vf1*(np.cos(alpha_L - theta1)* np.cos(self.delta1-epsilon
            ↪ ))/ (np.cos(self.delta1)*np.cos(self.delta4))


        dydt = np.zeros_like(y)

        dydt[0] = (self.VL * np.sin(alpha_L - theta1 - self.delta1))
            ↪ / (R1 * np.cos(self.delta1))
        dydt[1] = (self.VL * np.sin(alpha_L - theta2 - self.delta2))
            ↪ / (R2 * np.cos(self.delta2))
        dydt[2] = (self.VL * np.sin(alpha_L - theta3 - self.delta3))
            ↪ / (R3 * np.cos(self.delta3))
        dydt[3] = (Vf1 * np.sin(self.delta1- self.delta4-epsilon)) /
            ↪ (R4 * np.cos(self.delta4))

        dydt[4] = 0
```

```
        dydt[5] = dydt[0]
        dydt[6] = dydt[1]
        dydt[7] = dydt[2]
        dydt[8] = dydt[3]

        dydt[9] = self.VL * np.cos(alpha_L)
        dydt[10] = self.VL * np.sin(alpha_L)
        dydt[11] = Vf1 * np.cos(alpha_f1)
        dydt[12] = Vf1 * np.sin(alpha_f1)
        dydt[13] = Vf2 * np.cos(alpha_f2)
        dydt[14] = Vf2 * np.sin(alpha_f2)
        dydt[15] = Vf3 * np.cos(alpha_f3)
        dydt[16] = Vf3 * np.sin(alpha_f3)
        dydt[17] = Vf4 * np.cos(alpha_f4)
        dydt[18] = Vf4 * np.sin(alpha_f4)

        return dydt
```

Listing A.7: Movement by 'T' using DDP

```
    def moveM(self):
        self.VL = 0.3
        self.delta1=45*np.pi/180
        self.R=0.6*np.sqrt(2)
        self.y=[self.alphas['cf1'],self.alphas['cf2'],self.alphas['
            ↪ cf3'],self.alphas['cf4'],self.alphas['cf5'],self.alphas
            ↪ ['cf6'],self.alphas['cf7'],self.positions['cf1'][0]+
            ↪ self.R*np.cos(self.delta1),self.positions['cf1'][1],
            ↪ self.positions['cf2'][0],self.positions['cf2'][1],self.
            ↪ positions['cf3'][0],self.positions['cf3'][1],self.
            ↪ positions['cf4'][0],self.positions['cf4'][1],self.
            ↪ positions['cf5'][0],self.positions['cf5'][1],self.
            ↪ positions['cf6'][0],self.positions['cf6'][1],self.
            ↪ positions['cf7'][0],self.positions['cf7'][1]]

```

```
        t_span = (0,5)
        x_pts = np.linspace(0, 5, 101)
        sol = solve_ivp(self.kinematics_M, t_span, self.y,t_eval=
            ↪ x_pts)
        num=len(sol.y[0])
        for n in range(num):
            column = [row[n] for row in sol.y]

            pos={}
            pos['cf1']=[column[7]-self.R*np.cos(self.delta1),column
                ↪ [8],1.0]
            pos['cf2']=[column[9],column[10],1.0]
            pos['cf3']=[column[11],column[12],1.0]
            pos['cf4']=[column[13],column[14],1.0]
            pos['cf5']=[column[15],column[16],1.0]
            pos['cf6']=[column[17],column[18],1.0]
            pos['cf7']=[column[19],column[20],1.0]

            cfs={f'cf{i}': i-1 for i in range(1,8)}

            for cfName in cfs:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo(pos[cfName],0,0.8)
            self.swarm.timeHelper.sleep(0.06)

    def kinematics_M(self,t,y):
        self.delta1=45*np.pi/180
        self.delta2=-45*np.pi/180
        self.delta3=0.244978663
        self.delta4=0
        self.delta5=-0.463647609
        self.delta6=0

        R1=0.6*np.sqrt(2)
```

```
        R2=0.6*np.sqrt(2)
        R3=0.3*np.sqrt(17)
        R4=1.2
        R5=0.3*np.sqrt(5)
        R6=1.2

        theta1,theta2,theta3,theta4,theta5,theta6=-self.delta1,-self.
            ↪ delta2,-self.delta3,-self.delta4,-self.delta5,-self.
            ↪ delta6
        alpha_L, alpha_f1, alpha_f2, alpha_f3,alpha_f4,alpha_f5,
            ↪ alpha_f6 = y[0],y[1], y[2],y[3],y[4],y[5],y[6]

        xl,yl=y[7],y[8]
        x1,y1=y[9],y[10]
        x2,y2=y[11],y[12]
        x3,y3=y[13],y[14]
        x4,y4=y[15],y[16]
        x5,y5=y[17],y[18]
        x6,y6=y[19],y[20]

        epsilon4=theta4-theta1
        epsilon5=theta5-theta3
        epsilon6=theta6-theta2
        Vf1 = self.VL * np.cos(alpha_L - theta1) / np.cos(self.delta1
            ↪ )
        Vf2 = self.VL * np.cos(alpha_L - theta2) / np.cos(self.delta2
            ↪ )
        Vf3 = self.VL * np.cos(alpha_L - theta3) / np.cos(self.delta3
            ↪ )
        Vf4=Vf1*(np.cos(alpha_L - theta1)* np.cos(self.delta1-
            ↪ epsilon4))/ (np.cos(self.delta1)*np.cos(self.delta4))
        Vf5=Vf3*(np.cos(alpha_L - theta3)* np.cos(self.delta3-
            ↪ epsilon5))/ (np.cos(self.delta3)*np.cos(self.delta5))
        Vf6=Vf2*(np.cos(alpha_L - theta2)* np.cos(self.delta2-
```

```
            ↪ epsilon6))/ (np.cos(self.delta2)*np.cos(self.delta6))
65
66
67          dydt = np.zeros_like(y)
68
69          dydt[0] = 0
70          dydt[1] = (self.VL * np.sin(alpha_L - theta1 - self.delta1))
               ↪ / (R1 * np.cos(self.delta1))
71          dydt[2] = (self.VL * np.sin(alpha_L - theta2 - self.delta2))
               ↪ / (R2 * np.cos(self.delta2))
72          dydt[3] = (self.VL * np.sin(alpha_L - theta3 - self.delta3))
               ↪ / (R3 * np.cos(self.delta3))
73          dydt[4] = (Vf1 * np.sin(self.delta1- self.delta4-epsilon4)) /
               ↪  (R4 * np.cos(self.delta4))
74          dydt[5] = (Vf3 * np.sin(self.delta3- self.delta5-epsilon5)) /
               ↪  (R5 * np.cos(self.delta5))
75          dydt[6] = (Vf2 * np.sin(self.delta2- self.delta6-epsilon6)) /
               ↪  (R6 * np.cos(self.delta6))
76
77
78          dydt[7] = self.VL * np.cos(alpha_L)
79          dydt[8] = self.VL * np.sin(alpha_L)
80          dydt[9] = Vf1 * np.cos(alpha_f1)
81          dydt[10] = Vf1 * np.sin(alpha_f1)
82          dydt[11] = Vf2 * np.cos(alpha_f2)
83          dydt[12] = Vf2 * np.sin(alpha_f2)
84          dydt[13] = Vf3 * np.cos(alpha_f3)
85          dydt[14] = Vf3 * np.sin(alpha_f3)
86          dydt[15] = Vf4 * np.cos(alpha_f4)
87          dydt[16] = Vf4 * np.sin(alpha_f4)
88          dydt[17] = Vf5 * np.cos(alpha_f5)
89          dydt[18] = Vf5 * np.sin(alpha_f5)
90          dydt[19] = Vf4 * np.cos(alpha_f6)
91          dydt[20] = Vf4 * np.sin(alpha_f6)
```

```
92
93          return dydt
```

Listing A.8: Movement by 'M' using DDP

A.5 RETURN

```
1  X=np.array([
2              [self.positions[cf][0],self.positions[cf][1],1.0] for cf
                   ↪ in self.swarm.allcfs.crazyfliesByName
3          ])
4
5          n_agents = 7
6          dim = 3
7          dt = 0.05
8          max_speed = 0.3
9          collision_radius = 0.28
10         goal_tolerance = 0.01
11
12         V = np.zeros_like(X)
13         history = [X.copy()]
14
15         def all_reached(X, X_target, tol=goal_tolerance):
16             return np.all(np.linalg.norm(X - X_target, axis=1) < tol)
17
18
19         X_target = np.array([
20             [2.0, 0.2, 1.0],
21             [1.7, 0.2, 1.0],
22             [2.3, 0.2, 1.0],
23             [0.2,0.2,1.0],
24             [-0.1,0.2,0.5],
25             [-0.4,0.2,0.5],
26             [-0.7,0.2,0.5]
```

```
        ])

        def compute_vo_velocity(X, V, i, X_target):
            to_target = X_target[i] - X[i]
            dist = np.linalg.norm(to_target)
            desired = to_target / (dist + 1e-6) * max_speed if dist >
                ↪  1e-3 else np.zeros(3)

            for j in range(n_agents):
                if i != j:
                    rel_pos = X[j] - X[i]
                    d = np.linalg.norm(rel_pos)
                    if d < collision_radius:
                        avoidance = -rel_pos / (d + 1e-6) * max_speed
                        desired += avoidance

            speed = np.linalg.norm(desired)
            if speed > max_speed:
                desired = desired / speed * max_speed
            return desired

        max_iters = 2000
        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1
```

```
59
60
61          history = np.array(history)
62          history=history[::3]
63          step=len(history)
64
65          cfs={f'cf{i}': i-1 for i in range(1,4)}
66          print(cfs)
67
68
69
70          for i in range(step):
71              target=history[i]
72              t=5.0/step
73              print(target)
74              for cfName in cfs:
75                  cf=self.swarm.allcfs.crazyfliesByName[cfName]
76                  cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                        ↪ ]][1],1.0],0,0.8)
77                  print(f'{cfName}:{target[cfs[cfName]]}')
78              self.timeHelper.sleep(t)
79          print(step)
80          self.timeHelper.sleep(2.0)
81
82          X=np.array([
83              self.positions[cf] for cf in self.swarm.allcfs.
                    ↪ crazyfliesByName
84          ])
85
86          X_target = np.array([
87              [0.8, 0.2, 1.0],
88              [0.5, 0.2, 1.0],
89              [1.1, 0.2, 1.0],
90              [0.2,0.2,1.0],
```

```
            [-0.1,0.2,0.0],
            [-0.4,0.2,0.0],
            [-0.7,0.2,0.0]
        ])

        V = np.zeros_like(X)
        history = [X.copy()]

        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1



        history = np.array(history)
        history=history[::3]
        step=len(history)

        for i in range(step):
            target=history[i]
            t=5.0/step
            print(target)
            for cfName in cfs:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
```

```
                print(f'{cfName}:{target[cfs[cfName]]}')
            self.timeHelper.sleep(t)

        print(step)
```

Listing A.9: Return of 'I'

```
    def returnT(self):
        X=np.array([
            self.positions[cf] for cf in self.swarm.allcfs.
                ↪ crazyfliesByName
        ])

        n_agents = 7
        dim = 3
        dt = 0.05
        max_speed = 0.2
        collision_radius = 0.28
        goal_tolerance = 0.01

        V = np.zeros_like(X)
        history = [X.copy()]

        def all_reached(X, X_target, tol=goal_tolerance):
            return np.all(np.linalg.norm(X - X_target, axis=1) < tol)

        X_target = np.array([
            [2.2, 0.2, 1.0],
            [1.9, 0.2, 1.0],
            [1.6, 0.2, 1.0],
            [1.3,0.2,1.0],
            [1.0,0.2,1.0],
            [-0.4,0.2,1.0],
            [-0.7,0.2,1.0]
        ])
```

```
28
29      def compute_vo_velocity(X, V, i, X_target):
30          to_target = X_target[i] - X[i]
31          dist = np.linalg.norm(to_target)
32          desired = to_target / (dist + 1e-6) * max_speed if dist >
                ↪  1e-3 else np.zeros(3)
33
34          for j in range(n_agents):
35              if i != j:
36                  rel_pos = X[j] - X[i]
37                  d = np.linalg.norm(rel_pos)
38                  if d < collision_radius:
39                      avoidance = -rel_pos / (d + 1e-6) * max_speed
40                      desired += avoidance
41
42          speed = np.linalg.norm(desired)
43          if speed > max_speed:
44              desired = desired / speed * max_speed
45          return desired
46
47      max_iters = 200
48      step = 0
49
50      while not all_reached(X, X_target) and step < max_iters:
51          new_V = np.zeros_like(X)
52          for i in range(n_agents):
53              if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
54                  new_V[i] = compute_vo_velocity(X, V, i, X_target)
55          X += dt * new_V
56          V = new_V
57          history.append(X.copy())
58          step += 1
59
```

```
60
61          history = np.array(history)
62          history=history[::3]
63          step=len(history)
64
65          cfs={f'cf{i}': i-1 for i in range(1,6)}
66          print(cfs)
67
68
69
70          for i in range(step):
71              target=history[i]
72              t=5.0/step
73              # print(target)
74              for cfName in cfs:
75                  cf=self.swarm.allcfs.crazyfliesByName[cfName]
76                  cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                        ↪ ]][1],1.0],0,0.8)
77                  print(f'{cfName}:{target[cfs[cfName]]}')
78              self.timeHelper.sleep(t)
79
80
81          self.timeHelper.sleep(2.0)
82
83          X=np.array([
84              self.positions[cf] for cf in self.swarm.allcfs.
                    ↪ crazyfliesByName
85          ])
86
87          X_target = np.array([
88              [1.1, 0.2, 1.0],
89              [0.8, 0.2, 1.0],
90              [0.5, 0.2, 1.0],
91              [0.2,0.2,1.0],
```

```
            [-0.1,0.2,1.0],
            [-0.4,0.2,1.0],
            [-0.7,0.2,1.0]
        ])

        V = np.zeros_like(X)
        history = [X.copy()]

        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1



        history = np.array(history)
        history=history[::3]
        step=len(history)
        for i in range(step):
            target=history[i]
            t=5.0/step
            # print(target)
            for cfName in cfs:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
                print(f'{cfName}:{target[cfs[cfName]]}')
```

```
            self.timeHelper.sleep(t)
```

Listing A.10: Return of 'T'

```
    def returnM(self):
        X=np.array([
            self.positions[cf] for cf in self.swarm.allcfs.
                ↪ crazyfliesByName
        ])

        n_agents = 7
        dim = 3
        dt = 0.05
        max_speed = 0.3
        collision_radius = 0.28
        goal_tolerance = 0.01

        V = np.zeros_like(X)
        history = [X.copy()]

        def all_reached(X, X_target, tol=goal_tolerance):
            return np.all(np.linalg.norm(X - X_target, axis=1) < tol)

        X_target = np.array([
            [2.3, 0.2, 1.0],
            [2.0, 0.2, 1.0],
            [1.7, 0.2, 1.0],
            [1.4,0.2,1.0],
            [1.1,0.2,1.0],
            [0.8,0.2,1.0],
            [0.5,0.2,1.0]
        ])

        def compute_vo_velocity(X, V, i, X_target):
            to_target = X_target[i] - X[i]
```

```
            dist = np.linalg.norm(to_target)
            desired = to_target / (dist + 1e-6) * max_speed if dist >
                ↪  1e-3 else np.zeros(3)

            for j in range(n_agents):
                if i != j:
                    rel_pos = X[j] - X[i]
                    d = np.linalg.norm(rel_pos)
                    if d < collision_radius:
                        avoidance = -rel_pos / (d + 1e-6) * max_speed
                        desired += avoidance

            speed = np.linalg.norm(desired)
            if speed > max_speed:
                desired = desired / speed * max_speed
            return desired

        max_iters = 2000
        step = 0

        while not all_reached(X, X_target) and step < max_iters:
            new_V = np.zeros_like(X)
            for i in range(n_agents):
                if np.linalg.norm(X[i] - X_target[i]) >=
                    ↪ goal_tolerance:
                    new_V[i] = compute_vo_velocity(X, V, i, X_target)
            X += dt * new_V
            V = new_V
            history.append(X.copy())
            step += 1


        history = np.array(history)
        history=history[::3]
```

```
        step=len(history)

        cfs={f'cf{i}': i-1 for i in range(1,8)}
        print(cfs)




        for i in range(step):
            target=history[i]
            t=5.0/step
            # print(target)
            for cfName in cfs:
                cf=self.swarm.allcfs.crazyfliesByName[cfName]
                cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                    ↪ ]][1],1.0],0,0.8)
                print(f'{cfName}:{target[cfs[cfName]]}')
            self.timeHelper.sleep(t)

        self.timeHelper.sleep(2.0)

        X=np.array([
            self.positions[cf] for cf in self.swarm.allcfs.
                ↪ crazyfliesByName
        ])

         X_target = np.array([
            [1.1,0.2,1.0],
            [0.8, 0.2, 1.0],
            [0.5, 0.2, 1.0],
            [0.2, 0.2, 1.0],
            [-0.1,0.2,1.0],
            [-0.4,0.2,1.0],
            [-0.7,0.2,1.0]
        ])
```

```
95
96          V = np.zeros_like(X)
97          history = [X.copy()]
98
99          step = 0
100
101         while not all_reached(X, X_target) and step < max_iters:
102             new_V = np.zeros_like(X)
103             for i in range(n_agents):
104                 if np.linalg.norm(X[i] - X_target[i]) >=
                        ↪ goal_tolerance:
105                     new_V[i] = compute_vo_velocity(X, V, i, X_target)
106             X += dt * new_V
107             V = new_V
108             history.append(X.copy())
109             step += 1
110
111
112         history = np.array(history)
113         history=history[::3]
114         step=len(history)
115
116         for i in range(step):
117             target=history[i]
118             t=5.0/step
119             # print(target)
120             for cfName in cfs:
121                 cf=self.swarm.allcfs.crazyfliesByName[cfName]
122                 cf.goTo([target[cfs[cfName]][0],target[cfs[cfName
                            ↪ ]][1],1.0],0,0.8)
123                 print(f'{cfName}:{target[cfs[cfName]]}')
124             self.timeHelper.sleep(t)
```

Listing A.11: Return of 'M'

BIBLIOGRAPHY

[1] Ouyang, Q., et al. (2023). Formation control of unmanned aerial vehicle swarms: A comprehensive review.

[2] Wang, Y., et al. (2024). Unmanned aerial vehicle formation control method based on improved artificial potential field and consensus.

[3] Cai, Z., et al. (2022). Virtual structure and artificial potential field-based cooperative control for UAV formation.

[4] Zhang, J., et al. (2019). A Formation Control Method for AUV Group Under Communication Delay.

[5] Mesbahi, M., & Egerstedt, M. (2010). Graph Theoretic Methods in Multiagent Networks. Princeton University Press.

[6] Li, J., et al. (2023). Leader-follower formation of light-weight UAVs with novel active disturbance rejection control.

[7] Nguyen, X.-M., & Hong, S. K. (2019). Robust adaptive formation control of quadcopters based on a leader–follower approach.

[8] MDPI. (2023). Robust Leader–Follower Formation Control Using Neural Adaptive Prescribed Performance Strategies.

[9] MDPI. (2023). Robust Hierarchical Formation Control of Unmanned Aerial Vehicles via Neural-Based Observers.

[10] Shay Segal, Joseph Z., Ben-Asher & Haim Weiss (2005), Derivation of Formation-Flight Guidance Lawsfor Unmanned Air Vehicles, Journal of Guidance, Control and Dynamics.